String Matching

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

August 29, 2025

Overview

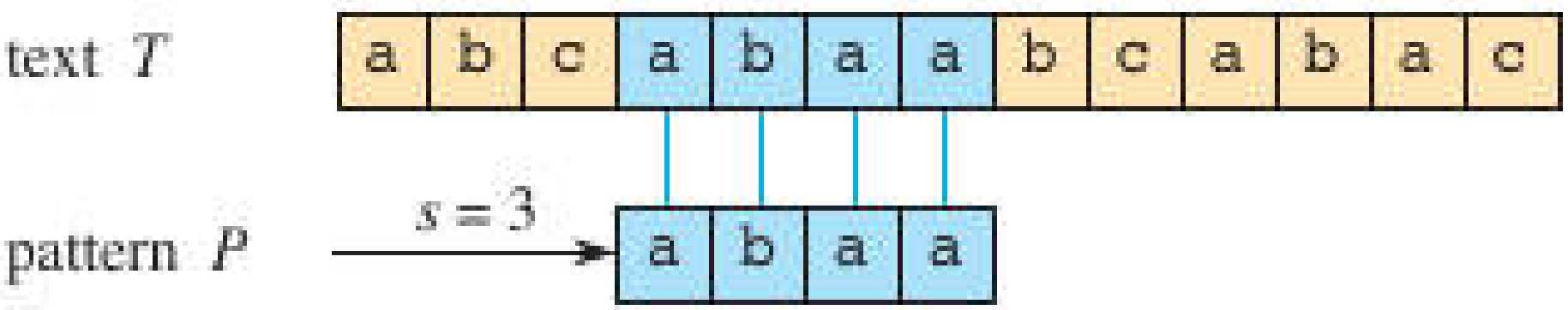
1. Preliminaries
2. Naive string Matching
3. String matching with Finite Automata
4. Knuth Morris Pratt Algorithm
5. Books etc.

Table of Contents

1. Preliminaries
2. Naive string Matching
3. String matching with Finite Automata
4. Knuth Morris Pratt Algorithm
5. Books etc.

String Matching Problem

- The text is given as an array $T[1..n]$ of length n, and the pattern is an array $P[1..m]$ of length $m \leq n$.
- The elements of P and T are characters drawn from an alphabet Σ, which is a finite set of characters. For example, $\Sigma = \{a, b, \cdots, z\}$. The character arrays P and T are strings of characters
- Pattern P occurs with **shift** s in text T (or, equivalently, that pattern P occurs beginning at position $s+1$ in text T) if $0 \leq s \leq n-m$ and $T[s+1 : s+m] = P[1 : m]$, that is, if $T[s+j] = P[j]$, for $1 \leq j \leq m$. Here P occurs with shift s in T, thus s is a **valid shift**.
- **The string-matching problem** is the problem of finding **all valid shifts** with which a given pattern P occurs in a given text T.



String Matching Problem: Terminology

- We denote by $\Sigma*$ the set of all finite-length strings formed using characters from the alphabet Σ. **Empty String**: ϵ
- **Concatenation** of two strings x and y , denoted xy , has length $|x| + |y|$ and consists of the characters from x followed by the characters from y.
- A string w is a **prefix** of a string x, denoted $w \sqsubset x$, if $x = wy$ for some string $y \in \Sigma*$. if $w \sqsubset x$, then $|w| \leq |x|$.
- A string w is a **suffix** of a string x , denoted $w \sqsupset x$, if $x = yw$ for some $y \in \Sigma*$. $w \sqsupset x$ implies $|w| \leq |x|$. **Example:** $ab \sqsubset \underline{ab}cca$ and $cca \sqsupset ab\underline{cca}$.
- A string w is a **proper prefix** of x if $w \sqsubset x$ and $|w| < |x|$, and likewise a **proper suffix** is defined. The empty string ϵ is both a suffix and a prefix of every string.
- Denote k -character preffix $P[1:k]$ of the pattern $P[1:m] = P = P[:m]$ by $P[:k]$ or P_k.
- Similarly, denote the k -character preffix of the text T by $T[:k]$
- Thus, **string-matching problem** is finding all shifts s in the range $0 \leq s \leq n - m$ such that $P \sqsupset T[:s+m]$.

String Matching Problem: Terminology

Overlapping Suffix Lemma: Suppose that x, y, and z are strings such that $x \sqsupset z$ and $y \sqsupset z$. (a) If $|x| \leq |y|$, then $x \sqsupset y$. (b) If $|x| \geq |y|$, then $y \sqsupset x$. (c) If $|x| = |y|$, then $x = y$.

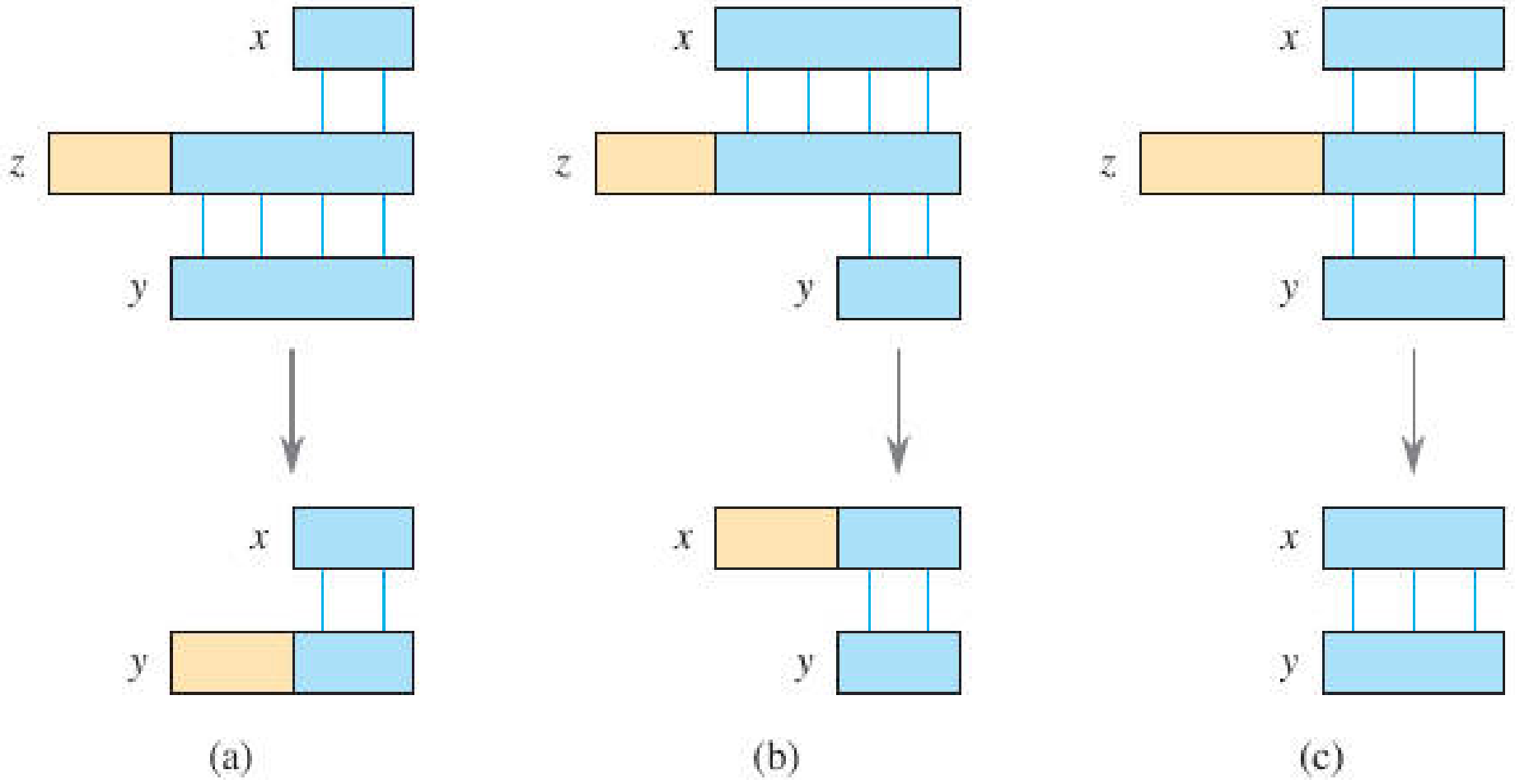
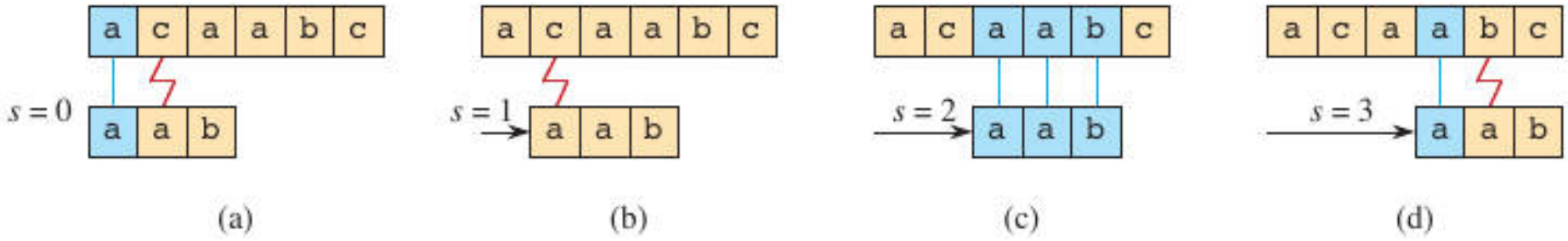


Table of Contents

1. Preliminaries
2. Naive string Matching
3. String matching with Finite Automata
4. Knuth Morris Pratt Algorithm
5. Books etc.

Naive string matching algorithm

```
NAIVE-STRING-MATCHER(T, P, n, m)
for s = 0 to n − m
    if P[1 : m] == T[s + 1 : s + m]
        print "Pattern occurs with shift" s
```



Time complexity: $\Theta(n - m + 1)m)$, if $m = n/2$, then $\Theta(n^2)$

Table of Contents

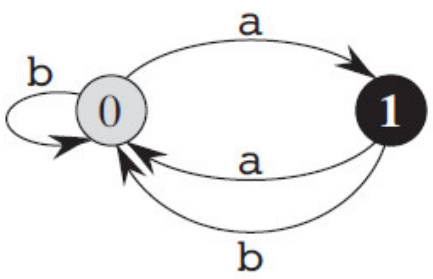
1. Preliminaries
2. Naive string Matching
3. String matching with Finite Automata
4. Knuth Morris Pratt Algorithm
5. Books etc.

String Matching with Finite Automata (1)

Finite Automata: a **finite automata M**, is a 5-tuple $(Q, q_0, A, \Sigma, \delta)$, where

- Q: finite set of **states**
- $q_0 \in Q$ is the **start state**
- $A \subseteq Q$ is distinguished set of **accepting states**
- Σ is finite **input alphabet**
- δ is a function $Q \times \Sigma \to Q$, **transition function of M**

state	input a	input b
0	1	0
1	0	0

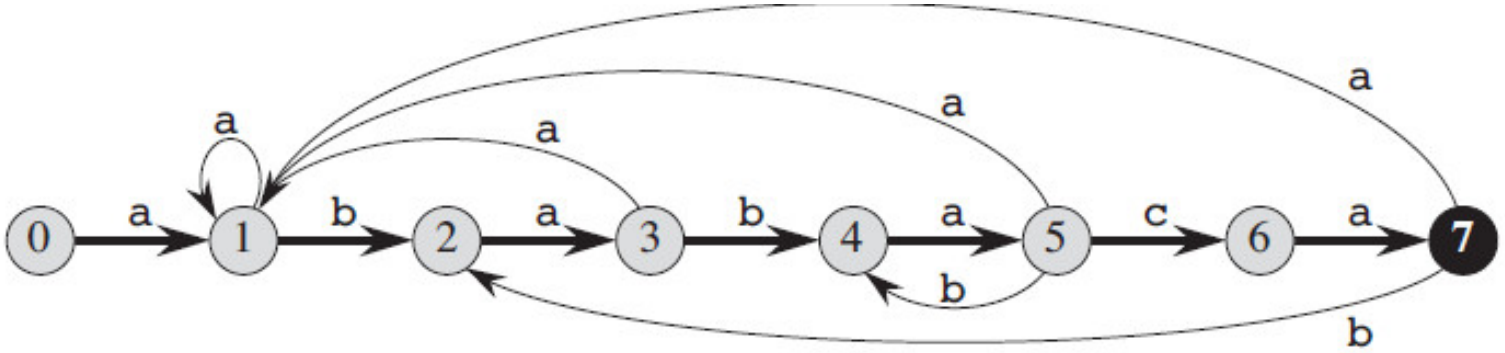


String Matching with Finite Automata (2)

- FA begins with q_0 and reads character of input string one at a time.
- if automata is in state q and next input is a, it goes to a state defined in table $\delta(q, a)$.
- if $q \in A$, M has accepted the string read so far.
- M induces function ϕ, **final state function**, $\Sigma^* \to Q$ such that $\phi(w)$ ends up after reading string w
- M accepts a string w iff $\phi(w) \in A$
- $\phi(\epsilon) = q_0$; $\phi(wa) = \delta(\phi(w), a)$ for $w \in \Sigma^*, a \in \Sigma$

String Matching Automata

- For a given Pattern P, construct string matching automata as **preprocessing step**



state	input a	input b	input c	P
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

i	—	1	2	3	4	5	6	7	8	9	10	11
$T[i]$	—	a	b	a	b	a	b	a	c	a	b	a
state $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

Suffix function σ

- σ function is defined for a given pattern $P[1..m]$
- $\sigma : \Sigma^* \rightarrow \{0, 1, \cdots, m\}$ such that $\sigma(x)$ is length of **longest prefix of** P that is also a **suffix of** x
- $\sigma(x) = max\{k : P_k \sqsupset x\}$
- example 1: $P_0 = \epsilon$ suffix of any string. $P = ab$, $\sigma(\epsilon) = 0$
- example 2 and 3: $P = \underline{a}b$ $\sigma(ccac\underline{a}) = 1$ and $\sigma(cc\underline{ab}) = 2$ for $P = \underline{ab}$
- for P of length m, $\sigma(x) = m$ iff $P \sqsupset x$
- if $x \sqsupset y$ then $\sigma(x) \leq \sigma(y)$

Defining String matching automaton for a pattern P (1)

- $Q = \{0, 1, \cdots, m\}$; state m is the only accepting state
- $\delta(q, a) = \sigma(P_q a)$ for any state q and input character a; keep track of longest prefix of pattern P that has matched Text string T_i so far.

Justification for δ:

- let $T[i]$ is just read to match P_j; i.e, prefix P_j must be suffix of T_i.
- $q = \phi(T_i)$, the current state after reading T_i
- design δ such that q tells the length of longest prefix of P that matches a suffix of T_i, thus we find a match.
- state q implies $P_q \sqsupset T_i$ and $q = \sigma(T_i)$; if $q == m$, P matches with suffix of T_i. thus match found; i.e $\phi(T_i) = \sigma(T_i)$
- let automata in state q, reads $T[i+1] = a$, then transition should be to a state that is longest prefix of P that is a suffix of $T_i a$; state $\sigma(T_i a)$
- since P_q is longest prefix of P that is suffix of T_i, the longest prefix of P, that is suffix of $T_i a$ is $\sigma(T_i a) = \sigma(P_q a)$,as longest prefix of P is also a suffix of $P_q a$
- Thus $\delta(q, a) = \sigma(P_q a)$

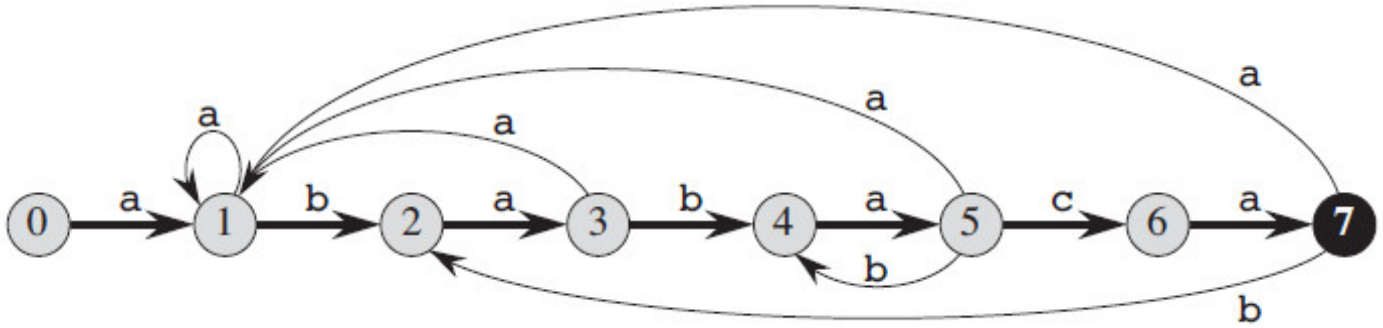
Defining String matching automaton for a pattern P (2)

Two cases:

- $a = P[q+1]$: character a <u>matches</u> the $q+1^{th}$ character in Pattern; so, $\delta(q, a) = q+1$; transition continues along *spine* of automaton
- $a \neq P[q+1]$: character a <u>does not match</u> the $q+1^{th}$ character in Pattern; so find smaller prefix of P that is also suffix of T_i

Example:

- $\delta(5, c) = 6; c = P[6]$: character c <u>matches</u> the 6th character c of Pattern; transition continues along *spine* of automaton
- $\delta(5, b) = 4; c = P[6]$: character b <u>does not match</u> the 6th character c in Pattern; so find smaller prefix of P that is also suffix of T_i, ie, $P_q b =$ ababab, and the longest prefix of P (next state q) that is also a suffix of *ababab* , ie, $P_4 = abab$



Finite Automaton Matcher

```
FINITE-AUTOMATON-MATCHER(T, δ, m)
n = T.length
q = 0
for i = 1 to n
    q = δ(q, T[i])
    if q == m
        print "Pattern occurs with shift" i − m
```

- After scanning $T[i]$, machine is in state $\sigma(T_i)$
- $\sigma(T_i) = m$ iff $P \sqsupset T_i$; the machine is in accepting state m iff it has scanned pattern P.
- Time complexity: without preprocessing, it takes $\Theta(n)$ time, where n: length of text.

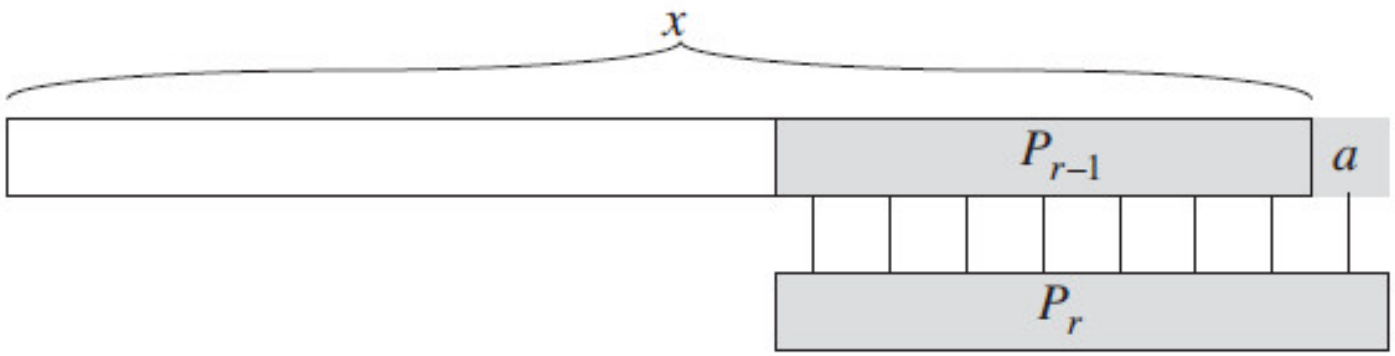
Lemma 1: Suffix function inequality

Lemma 1: For any string x and character a, $\sigma(xa) \leq \sigma(x) + 1$.

- Example: $x = abababa$, $P = ababa$, $\sigma(xb) = \sigma(ababab\underline{a}) = 4$ and
- $\sigma(x) = \sigma(abababa) = 5$.
- thus, $\sigma(ababab\underline{a}) = 4 \leq (\sigma(x) + 1) = \sigma(abababa) + 1 = 5 + 1 = 6$

Proof:

- let $r = \sigma(xa)$, if $r = 0$ (no prefix), then $\sigma(xa) = 0 \leq \sigma(x) + 1$, as $\sigma(x)$ is non negative.
- if $r > 0$, then $P_r \sqsupset xa$ by definition of σ; then, $P_{r-1} \sqsupset x$
- So, $r - 1 \leq \sigma(x)$
- $\sigma(x)$ is longest k such that $P_k \sqsupset x$, thus, $\sigma(xa) = r \leq \sigma(x) + 1$

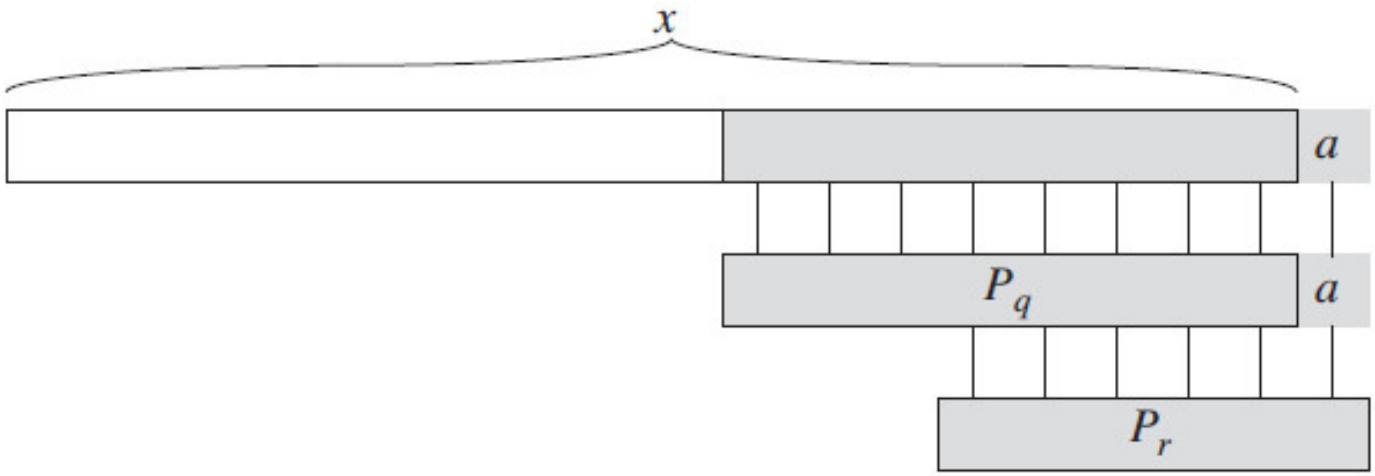


Lemma 2: Suffix function recursion

Lemma 2: For any string x and character a, if $q = \sigma(x)$, then $\sigma(xa) = \sigma(P_q a)$.

Proof:

- Definition of σ, $P_q \sqsupset x$, $P_q a \sqsupset xa$ (figure)
- let $r = \sigma(xa)$, ie, $P_r \sqsupset xa$ and from previous lemma $r \leq q + 1$
- thus, $|P_r| = r \leq q + 1 = |P_q a|$
- now $P_q a \sqsupset xa$, $P_r \sqsupset xa$ and $|P_r| \leq |P_q a|$, from overlapping suffix lemma, $P_r \sqsupset P_q a$
- so, $r \leq \sigma(P_q a)$, ie, $\sigma(xa) \leq \sigma(P_q a)$
- but, $\sigma(P_q a) \leq \sigma(xa)$, since $P_q a \sqsupset xa$. thus, $\sigma(xa) = \sigma(P_q a)$



Theorem

If ϕ is final state function of string matching automaton for a given pattern P and $T[1..n]$ is an input text, then $\phi(T_i) = \sigma(T_i)$ for $i = 0, 1, \cdots n$

Proof: by induction on i

- $i = 0$, $T_0 = \epsilon$, thus $\phi(T_0) = 0 = \sigma(T_0)$
- assume $\phi(T_i) = \sigma(T_i)$, to prove $\phi(T_{i+1}) = \sigma(T_{i+1})$
- let $q = \phi(T_i)$ and $a = T[i+1]$, then

$$
\begin{aligned}
\phi(T_{i+1}) &= \phi(T_i a) && \text{by definition of } T[i+1], \text{ a} \\
&= \delta(\phi(T_i), a) && \text{by definition of } \phi \\
&= \delta(q, a) && \text{by definition of } q \\
&= \sigma(P_q a) && \text{by definition } \delta(q, a) = \sigma(P_q a) \\
&= \sigma(T_i a) && \text{by Lemma 2 and induction} \\
&= \sigma(T_{i+1}) && \text{by definition of } T[i+1]
\end{aligned}
$$

Computing transition function

```
COMPUTE-TRANSITION-FUNCTION(P, Σ)
m = P.length
for q = 0 to m
    for each character a ∈ Σ
        k = min(m + 1, q + 2)
        repeat
            k = k − 1
        until P_k ⊐ P_q a
        δ(q, a) = k
return δ
```

- It starts with largest conceivable value of k. then it decreases k until $P_k \sqsupset P_q a$
- Time complexity: (two outer for loops contribute $m|\Sigma|$) $\times$ (inner loop runs atmost $m+1$ times $\times m$ time for $P_k \sqsupset P_q a$). Thus time complexity is $O(m^3|\Sigma|)$ where m: length of pattern.

Table of Contents

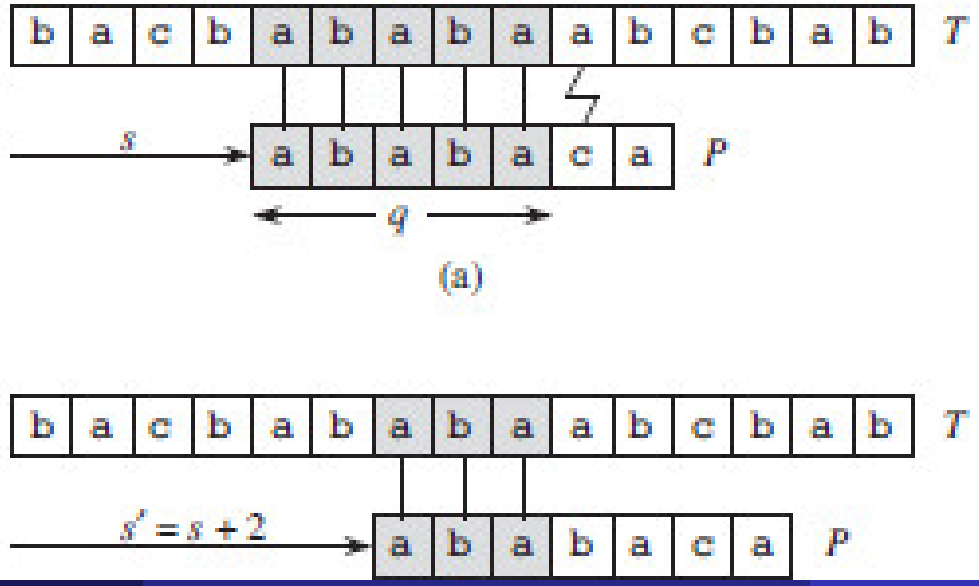
1. Preliminaries
2. Naive string Matching
3. String matching with Finite Automata
4. Knuth Morris Pratt Algorithm
5. Books etc.

KMP overview

- **No computation of time inefficient δ or transition function**, but precomputes a $\Theta(n)$ time **auxillary function** π for the given pattern P as a preprocessing step.
- Computed function π is stored in an array $\pi[1..m]$
- For state $q = 0, 1, \cdots m$ and any character $a \in \Sigma$, $\pi[q]$ stores information needed to compute $\delta(q, a)$ independent of a. thus saving a factor of $|\Sigma|$ in preprocessing step.

prefix function π for a pattern (1)

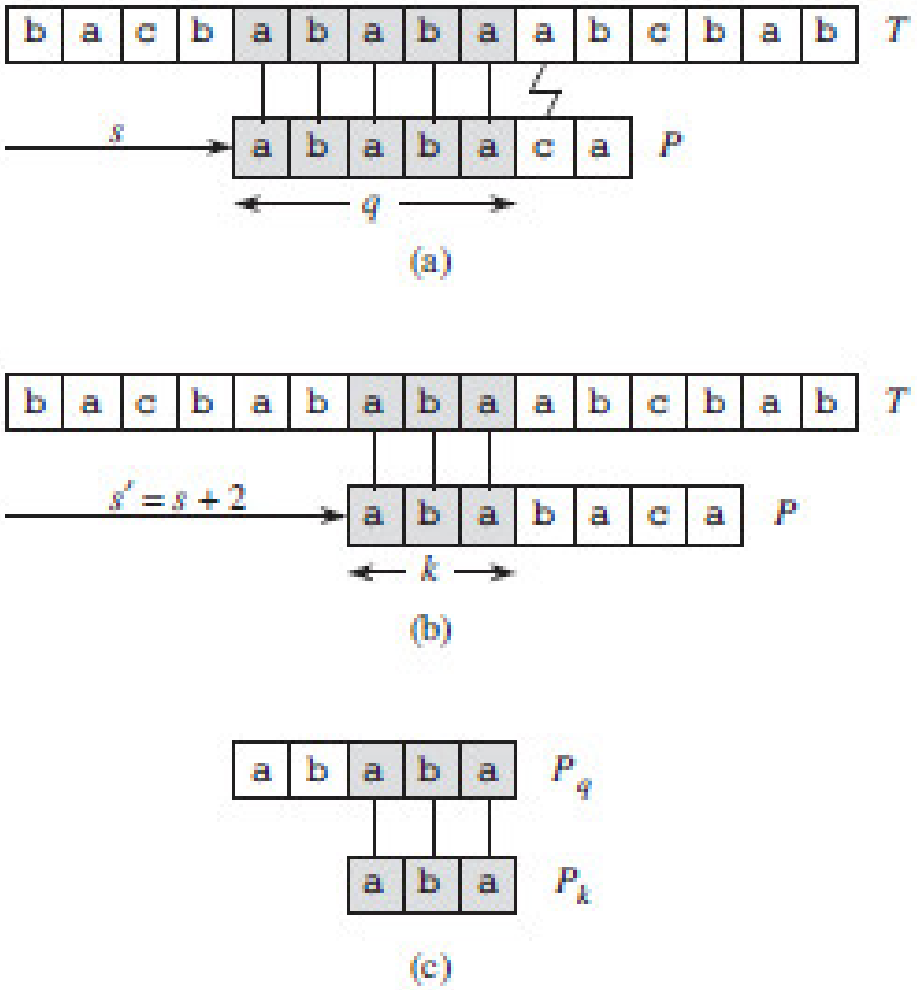
- captures information how the pattern matches against shifts of itself.
- when q characters of pattern matched against a Text, we know some shifts are invalid.
- Figure (a): $q = 5$ characters of pattern has matched text T with a shift s. Shifts $s + 1$ is invalid since $P_1 = a$ does not match with the next text character b, but text character b matches with second pattern character b.
- Figure(b): $s' = s + 2$ aligns $P_3 = aba$ with 3 text character that must neccessarily match.



prefix function π for a pattern (2)

Problem: Given that pattern $P[1..q]$ match text characters $T[s+1..s+q]$, what is the least shift $s' > s$ such that for some $k < q$, $P[1..k] = T[s'+1..s'+k]$, where $s'+k = s+q$?

- Knowing $P_q \sqsupset T_{s+q}$, need the longest **proper** prefix P_k of P_q that is also suffix of T_{s+q}; $s' = s + (q-k)$.
- If $k = 0$ means no prefix of P_k is suffix of $T[s+1..s+q]$, ie, safely we can skip checking $s+1$ to $s+q$ characters of T, thus $s' = s+q$
- Thus, precompute information by comparing the pattern against itself.(Figure (c)).



prefix function π for a pattern (3)

- Note $T[s'+1..s'+k]$ is part of known portion of text and also suffix of string P_q.
- **Problem reconstructed**: find the greatest $k < q$ such that $P_k \sqsupset P_q$.
- new shift $s' = s + (q - k)$ is next potentially valid shift.
- For each value of q, store the number k of matching characters at new shift s' in $\pi[q]$
- $\pi[q]$ is the length of longest prefix of P that is *proper* suffix of P_q

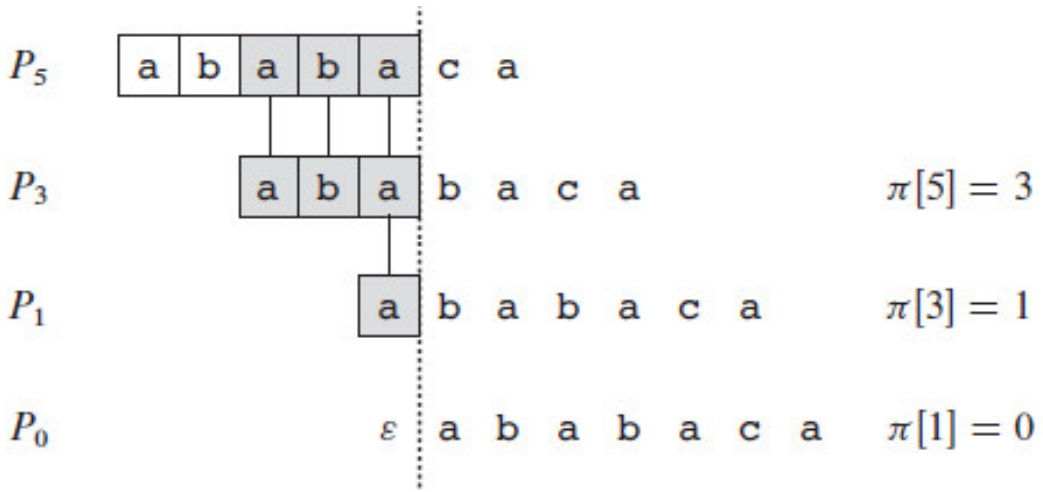
Prefix function: Given a pattern $P[1..m]$, the prefix function for pattern P is function $\pi : \{1, 2, \cdots, m\} \rightarrow \{0, 1, 2, \cdots, m-1\}$ such that

$$\pi[q] = max\{k : k < q \text{ and } P_k \sqsupset P_q\}$$

Example: prefix function computation

- let $q = 5$, slide the template of pattern P to right to note when some **prefix** P_k of P matches with some **proper suffix of** P_5
- matches for $k = 3, 1, 0 < q = 5$
- $\{k : k < 5 \text{and} P_k \sqsupset P_5\} = \{3, 1, 0\}$

i	1	2	3	4	5	6	7
$P[i]$	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1



Compute Prefix Function

```
COMPUTE-PREFIX-FUNCTION(P)
m = P.length
let π[1..m] be a new array
π[1] = 0
k = 0
for q = 2 to m
    while k > 0 and P[k + 1] ≠ P[q]
        k = π[k]
    if P[k + 1] == P[q]
        k = k + 1
    π[q] = k
return π
```

Compute Prefix Function: time complexity

```
COMPUTE-PREFIX-FUNCTION(P)
1   m = P.length
2   let π[1..m] be a new array
3   π[1] = 0
4   k = 0
5   for q = 2 to m
6       while k > 0 and P[k+1] ≠ P[q]
7           k = π[k]
8       if P[k+1] == P[q]
9           k = k+1
10      π[q] = k
11  return π
```

- Time complexity: $\Theta(m)$ by amortized analysis.
- a) k starts at 0, and k can increase only in line 9, which executes at most once per iteration of for loop
- thus, k can increase only upto $m-1$
- b) $k < q$ as it enters for loop and q is increased once in each iteration, ie, always $k < q$
- thus, line 3 and 10 ensures $\pi[q] < q$ for all $q = 1, 2, \cdots, m$, ie, while loop only decreases k
- c) k is never negative
- from a,b,c , total decrease in k in while loop is bounded from above by total increase in k over all iteration of for loop, ie, $m-1$
- overall while iterates atmost $m-1$ times.
- Thus, $\Theta(m)$

KMP Matcher

```
KMP-MATCHER(T, P)
n = T.length
m = P.length
π = COMPUTE-PREFIX-FUNCTION(P)
q = 0                                   // number of characters matched
for i = 1 to n                          // scan the text from left to right
    while q > 0 and P[q + 1] ≠ T[i]
        q = π[q]                        // next character does not match
    if P[q + 1] == T[i]
        q = q + 1                       // next character matches
    if q == m                           // is all of P matched?
        print "Pattern occurs with shift" i − m
        q = π[q]                        // look for the next match
```

- Time complexity: $\Theta(n)$ by amortized analysis.

Table of Contents

1. Preliminaries
2. Naive string Matching
3. String matching with Finite Automata
4. Knuth Morris Pratt Algorithm
5. Books etc.

References

Thomas H. Cormen,Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms, Fourth Edition, MIT Press, Chapter 32.1,32.3,32.4

The End